

zadania_lab_4

Zadanie 1

Napisz funkcję zwracającą wyznacznik macierzy kwadratowej dowolnego rozmiaru.

Jak rozumieć to zadanie?

Wyznacznik można policzyć tylko dla **macierzy kwadratowej**, czyli takiej, która ma tyle samo wierszy co kolumn.

Dla małych macierzy są proste wzory:

Dla macierzy 1×1

$$\det(A) = a_{11}$$

Dla macierzy 2×2

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

Dla większych macierzy

Stosujemy **rozwinięcie Laplace'a** względem pierwszego wiersza:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} M_{1j}$$

gdzie M_{1j} to minor, czyli wyznacznik macierzy powstałej po skreśleniu pierwszego wiersza i j -tej kolumny.

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz, czy macierz jest kwadratowa.

Krok 2

Jeśli macierz ma rozmiar 1×1 , zwracasz jedyny element.

Krok 3

Jeśli macierz ma rozmiar 2×2 , liczysz wyznacznik ze wzoru:

$$ad - bc$$

Krok 4

Dla większych macierzy:

- tworzysz minory,
- liczysz ich wyznaczniki rekurencyjnie,
- sumujesz wszystko ze znakami $+$ i $-$.

```
import math # importujemy moduł math, ponieważ później używamy funkcji math.pow()

def minor(macierz, usun_wiersz, usun_kolumne): # funkcja tworzy minor, czyli
macierz po usunięciu jednego wiersza i jednej kolumny
    wynik = [] # tworzymy pustą listę, do której będą dodawane wiersze nowej
macierzy

    for i in range(len(macierz)): # przechodzimy po wszystkich wierszach macierzy
        if i == usun_wiersz: # sprawdzamy, czy aktualny wiersz jest tym, który
        trzeba usunąć
            continue # pomijamy ten wiersz i przechodzimy do następnego

        nowy_wiersz = [] # tworzymy pusty wiersz dla nowej macierzy
        for j in range(len(macierz[i])): # przechodzimy po wszystkich kolumnach w
aktualnym wierszu
            if j == usun_kolumne: # sprawdzamy, czy aktualna kolumna jest tą,
którą trzeba usunąć
                continue # pomijamy ten element i przechodzimy do następnej
kolumny

            nowy_wiersz.append(macierz[i][j]) # dodajemy element do nowego
wiersza, jeśli nie leży w usuwanej kolumnie

        wynik.append(nowy_wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy macierz po usunięciu wskazanego wiersza i kolumny

def wyznacznik_macierzy(macierz): # funkcja liczy wyznacznik macierzy
n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie
jest kwadratowa") # zgłaszamy błąd, jeśli macierz nie jest kwadratowa

    if n == 1: # sprawdzamy przypadek macierzy 1 × 1
        return macierz[0][0] # wyznacznik macierzy 1 × 1 to jej jedyny element

    if n == 2: # sprawdzamy przypadek macierzy 2 × 2
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0] #
liczymy wyznacznik ze wzoru ad - bc

    det = 0 # ustawiamy początkową wartość wyznacznika na 0
    for j in range(n): # przechodzimy po kolejnych elementach pierwszego wiersza
        podmacierz = minor(macierz, 0, j) # tworzymy minor przez usunięcie
pierwszego wiersza i kolumny j
        det += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz) # dodajemy kolejny składnik rozwinięcia Laplace'a

    return det # zwracamy obliczony wyznacznik
```

```
macierz = [ # tworzymy macierz, dla której będzie liczony wyznacznik
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

print("Wyznacznik macierzy: ", wyznacznik_macierzy(macierz)) # wypisujemy wynik
działania funkcji wyznacznik_macierzy
```

Wynik

Dla macierzy

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy:

$$\det(A) = 66$$

Zadanie 2

Napisz funkcję zwracającą transpozycję macierzy dowolnego rozmiaru.

Co to jest transpozycja?

Transpozycja macierzy polega na zamianie:

- wierszy na kolumny,
- kolumn na wiersze.

Jeśli:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

to:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz liczbę wierszy i kolumn.

Krok 2

Tworzysz nową macierz wynikową.

Krok 3

Dla każdej kolumny starej macierzy tworzysz nowy wiersz.

Czyli element:

$$a_{ij}$$

staje się elementem:

$$a_{ji}$$

Kod

PYTHON

```
def transpozycja(macierz): # funkcja wykonuje transpozycję macierzy, czyli
    # zamienia wiersze na kolumny
    liczba_wierszy = len(macierz) # zapisujemy liczbę wierszy macierzy
    liczba_kolumn = len(macierz[0]) # zapisujemy liczbę kolumn macierzy, czyli
    # długość pierwszego wiersza

    wynik = [] # tworzymy pustą listę, do której będziemy dodawać wiersze macierzy
    # po transpozycji

    for j in range(liczba_kolumn): # przechodzimy po kolumnach starej macierzy
        nowy_wiersz = [] # tworzymy nowy wiersz macierzy wynikowej
        for i in range(liczba_wierszy): # przechodzimy po wierszach starej
            # macierzy
            nowy_wiersz.append(macierz[i][j]) # dodajemy element z kolumny starej
            # macierzy do nowego wiersza
        wynik.append(nowy_wiersz) # dodajemy gotowy nowy wiersz do macierzy
        # wynikowej

    return wynik # zwracamy macierz po transpozycji

macierz = [ # tworzymy macierz, którą będziemy transponować
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

print("Przed transpozycją macierzy:") # wypisujemy informację, że poniżej będzie
# macierz przed transpozycją
for wiersz in macierz: # przechodzimy po kolejnych wierszach macierzy
    print(wiersz) # wypisujemy aktualny wiersz macierzy

wynik = transpozycja(macierz) # wywołujemy funkcję transpozycja i zapisujemy wynik
# do zmiennej wynik

print("Po transpozycji macierzy:") # wypisujemy informację, że poniżej będzie
# macierz po transpozycji
for wiersz in wynik: # przechodzimy po kolejnych wierszach macierzy po
    # transpozycji
    print(wiersz) # wypisujemy aktualny wiersz macierzy wynikowej
```

Wynik

Dla macierzy:

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

transpozycja ma postać:

$$\begin{bmatrix} 2 & 0 & -3 \\ 4 & 2 & 3 \\ 6 & -1 & 3 \end{bmatrix}$$

Zadanie 3

Napisz funkcję znajdującą macierz odwrotną do macierzy kwadratowej dowolnego rozmiaru za pomocą:

- a) rozwinięcia Laplace'a
- b) metody Gaussa-Jordana

Zadanie 3a — macierz odwrotna metodą Laplace'a

Idea metody

Jeśli macierz A ma wyznacznik różny od zera, to macierz odwrotna istnieje i można ją policzyć ze wzoru:

$$A^{-1} = \frac{1}{\det(A)} \cdot Adj$$

gdzie:

- $\det(A)$ to wyznacznik,
- Adj to macierz dołączona, czyli transpozycja macierzy dopełnień algebraicznych.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz wyznacznik macierzy.

Krok 2

Jeśli wyznacznik jest równy 0, macierz nie ma odwrotności.

Krok 3

Dla każdego elementu liczysz minor i dopełnienie algebraiczne:

$$C_{ij} = (-1)^{i+j} \det(M_{ij})$$

Krok 4

Tworzysz macierz dopełnień algebraicznych.

Krok 5

Robisz jej transpozycję, czyli macierz dołączoną.

Krok 6

Dzielisz każdy element przez wyznacznik.

```
import math # importujemy moduł math, ponieważ używamy funkcji math.pow()

def minor(macierz, usun_wiersz, usun_kolumne): # funkcja tworzy minor, czyli
macierz po usunięciu wskazanego wiersza i kolumny
    wynik = [] # tworzymy pustą listę na macierz wynikową

    for i in range(len(macierz)): # przechodzimy po indeksach wszystkich wierszy
macierzy
        if i == usun_wiersz: # sprawdzamy, czy aktualny wiersz jest tym, który ma
zostać usunięty
            continue # pomijamy ten wiersz

        nowy_wiersz = [] # tworzymy pusty wiersz do nowej macierzy
        for j in range(len(macierz[i])): # przechodzimy po indeksach wszystkich
kolumn w aktualnym wierszu
            if j == usun_kolumne: # sprawdzamy, czy aktualna kolumna jest tą,
która ma zostać usunięta
                continue # pomijamy ten element
            nowy_wiersz.append(macierz[i][j]) # dodajemy element do nowego
wiersza, jeśli nie jest w usuwanej kolumnie

        wynik.append(nowy_wiersz) # dodajemy nowy wiersz do macierzy wynikowej

    return wynik # zwracamy minor macierzy

def wyznacznik_macierzy(macierz): # funkcja liczy wyznacznik macierzy
n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie
jest kwadratowa") # zgłaszamy błąd, jeśli macierz nie jest kwadratowa

    if n == 1: # sprawdzamy przypadek macierzy 1 × 1
        return macierz[0][0] # wyznacznik macierzy 1 × 1 to jej jedyny element

    if n == 2: # sprawdzamy przypadek macierzy 2 × 2
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0] #
liczymy wyznacznik ze wzoru ad - bc

    det = 0 # ustawiamy początkową wartość wyznacznika na 0
    for j in range(n): # przechodzimy po elementach pierwszego wiersza
        podmacierz = minor(macierz, 0, j) # tworzymy minor przez usunięcie
pierwszego wiersza i kolumny j
        det += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz) # dodajemy składnik rozwinięcia Laplace'a

    return det # zwracamy obliczony wyznacznik
```

```

def transpozycja(macierz): # funkcja wykonuje transpozycję macierzy
    liczba_wierszy = len(macierz) # zapisujemy liczbę wierszy macierzy
    liczba_kolumn = len(macierz[0]) # zapisujemy liczbę kolumn macierzy

    wynik = [] # tworzymy pustą macierz wynikową

    for j in range(liczba_kolumn): # przechodzimy po kolumnach starej macierzy
        nowy_wiersz = [] # tworzymy nowy wiersz macierzy po transpozycji
        for i in range(liczba_wierszy): # przechodzimy po wierszach starej
macierzy
            nowy_wiersz.append(macierz[i][j]) # dodajemy element z kolumny starej
macierzy do wiersza nowej macierzy
        wynik.append(nowy_wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy macierz po transpozycji

def zeros(n,m): # funkcja tworzy macierz zerową o wymiarach n x m
    macierz = [] # tworzymy pustą listę na macierz

    for i in range(n): # przechodzimy po liczbie wierszy
        wiersz = [] # tworzymy pusty wiersz
        for j in range(m): # przechodzimy po liczbie kolumn
            wiersz.append(0) # dodajemy zero do aktualnego wiersza
        macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy

    return macierz # zwracamy macierz zerową

def macierz_odwrotna_Laplace(macierz): # punkt a # funkcja liczy macierz odwrotną
    metodą dopełnień algebraicznych
    n = len(macierz) # zapisujemy rozmiar macierzy
    d = wyznacznik_macierzy(macierz) # liczymy wyznacznik macierzy

    if d == 0: # sprawdzamy, czy wyznacznik jest równy zero
        raise ValueError("Macierz jest osobliwa, nie ma odwrotności") # jeśli
wyznacznik jest zerowy, macierz nie ma odwrotności

    C = zeros(n, n) # tworzymy macierz dopełnień algebraicznych wypełnioną zerami

    for i in range(n): # przechodzimy po wierszach macierzy
        for j in range(n): # przechodzimy po kolumnach macierzy
            M = minor(macierz, i, j) # tworzymy minor przez usunięcie wiersza i
oraz kolumny j
            C[i][j] = math.pow(-1, i+j) * wyznacznik_macierzy(M) # liczymy
dopełnienie algebraiczne elementu a_ij

    Adj = transpozycja(C) # tworzymy macierz dołączoną, czyli transponujemy
macierz dopełnień algebraicznych

    wynik = zeros(n, n) # tworzymy pustą macierz wynikową wypełnioną zerami
    for i in range(n): # przechodzimy po wierszach macierzy wynikowej
        for j in range(n): # przechodzimy po kolumnach macierzy wynikowej
            wynik[i][j] = Adj[i][j] / d # dzielimy każdy element macierzy
dołączonej przez wyznacznik macierzy

```

```

    return wynik # zwracamy macierz odwrotną

macierz = [ # tworzymy macierz, dla której będziemy liczyć macierz odwrotną
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

wynik_Laplace = macierz_odwrotna_Laplace(macierz) # liczymy macierz odwrotną
metodą Laplace'a i zapisujemy wynik

print("Macierz odwrotna Laplace:") # wypisujemy opis wyniku
for wiersz in wynik_Laplace: # przechodzimy po kolejnych wierszach macierzy
    odwrotnej
    print(wiersz) # wypisujemy aktualny wiersz macierzy odwrotnej

```

Wynik

Dla macierzy:

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909091 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Zadanie 3b — macierz odwrotna metodą Gaussa-Jordana

Idea metody

Tworzy się macierz rozszerzoną:

$$[A \mid I]$$

a następnie wykonuje się operacje na wierszach tak, aby lewa część stała się macierzą jednostkową:

$$[I \mid A^{-1}]$$

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz macierz rozszerzoną:

- po lewej masz macierz A ,

- po prawej macierz jednostkową I .

Krok 2

Dla każdej kolumny ustawiasz jedynkę na przekątnej.

Krok 3

Wyzerowujesz pozostałe elementy w tej kolumnie.

Krok 4

Po zakończeniu prawa część jest macierzą odwrotną.

```
import math # importujemy moduł math, chociaż w tym kodzie nie jest bezpośrednio
używany

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b # funkcja liczy macierz
odwrotną metodą Gaussa-Jordana
    n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Macierz musi być kwadratowa") # zgłaszamy błąd,
jeśli macierz nie jest kwadratowa

    # macierz rozszerzona [A | I] # będziemy tworzyć macierz złożoną z macierzy A
oraz macierzy jednostkowej
    rozszerzona_macierz = [] # tworzymy pustą listę na macierz rozszerzoną

    for i in range(n): # przechodzimy po kolejnych wierszach macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy rozszerzonej

        # lewa strona: macierz A # najpierw wpisujemy elementy oryginalnej
macierzy
        for j in range(n): # przechodzimy po kolumnach macierzy A
            wiersz.append(macierz[i][j]) # dodajemy element z macierzy A do
aktualnego wiersza
            # wiersz.append(float(macierz[i][j])) # alternatywnie można byłoby od
razu zamienić elementy na liczby zmiennoprzecinkowe

        # prawa strona: macierz jednostkowa I # potem dopisujemy macierz
jednostkową po prawej stronie
        for j in range(n): # przechodzimy po kolumnach macierzy jednostkowej
            if i == j: # sprawdzamy, czy element leży na przekątnej głównej
                wiersz.append(1) # jeśli tak, wpisujemy 1
            else: # jeśli element nie leży na przekątnej głównej
                wiersz.append(0) # wpisujemy 0

        rozszerzona_macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
rozszerzonej

    # algorytm Gaussa-Jordana # zaczynamy przekształcanie macierzy rozszerzonej
    for i in range(n): # przechodzimy po kolejnych kolumnach głównych
        # jeśli na przekątnej jest 0, zamień wiersze # element główny nie może być
zerem
        if rozszerzona_macierz[i][i] == 0: # sprawdzamy, czy element główny jest
równy zero
            znaleziono = False # zakładamy, że jeszcze nie znaleziono wiersza do
zamiany
            for k in range(i+1, n): # szukamy niżej wiersza, który ma niezerowy
element w tej samej kolumnie
                if rozszerzona_macierz[k][i] != 0: # sprawdzamy, czy dany wiersz
```

```

ma niezerowy element
        rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i] # zamieniamy aktualny wiersz z
wybrany wierszem
        znaleziono = True # zapisujemy, że udało się znaleźć wiersz do
zamiany
        break # kończymy szukanie, bo zamiana została wykonana
    if not znaleziono: # sprawdzamy, czy nie udało się znaleźć
odpowiedniego wiersza
        raise ValueError("Macierz nie ma odwrotności") # jeśli nie ma
wiersza do zamiany, macierz nie ma odwrotności

    # dzielenie całego wiersza przez element główny # normalizujemy wiersz,
żeby na przekątnej otrzymać 1
    element_glowny = rozszerzona_macierz[i][i] # zapisujemy element główny z
przekątnej
    for j in range(2 * n): # przechodzimy po wszystkich kolumnach macierzy
rozszerzonej
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny
# dzielimy każdy element wiersza przez element główny

    # zerowanie pozostałych elementów w tej kolumnie # robimy zera nad i pod
elementem głównym
    for k in range(n): # przechodzimy po wszystkich wierszach
        if k != i: # pomijamy aktualny wiersz główny
            wspolczynnik = rozszerzona_macierz[k][i] # zapisujemy liczbę,
którą trzeba wyzerować
            for j in range(2 * n): # przechodzimy po wszystkich kolumnach
macierzy rozszerzonej
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j] # odejmujemy odpowiednią wielokrotność
wiersza głównego

    odwrotna = [] # tworzymy pustą listę na macierz odwrotną
    for i in range(n): # przechodzimy po wierszach macierzy rozszerzonej
        wiersz = [] # tworzymy pusty wiersz macierzy odwrotnej
        for j in range(n, 2 * n): # przechodzimy tylko po prawej części macierzy
rozszerzonej
            wiersz.append(rozszerzona_macierz[i][j]) # dodajemy element z prawej
strony, czyli z macierzy odwrotnej
        odwrotna.append(wiersz) # dodajemy gotowy wiersz do macierzy odwrotnej

    return odwrotna # zwracamy obliczoną macierz odwrotną

macierz = [ # tworzymy macierz, dla której będziemy liczyć macierz odwrotną
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

wynik_Gauss_Jordan = macierz_odwrotna_Gaussa_Jordana(macierz) # wywołujemy
funkcję i zapisujemy wynik

```

```
print("Macierz odwrotna Gauss Jordan:") # wypisujemy opis wyniku
for wiersz in wynik_Gauss_Jordan: # przechodzimy po kolejnych wierszach macierzy
    print(wiersz) # wypisujemy aktualny wiersz macierzy odwrotnej
```

Wynik

Otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909083 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Porównanie wyników obu metod

Obie metody dają tę samą macierz odwrotną.

Mogą pojawić się bardzo małe różnice w zapisie dziesiętnym, ale wynik matematycznie jest ten sam.

Zadanie 4

Napisz funkcję, która wykona mnożenie dwóch macierzy.

Kiedy można mnożyć macierze?

Macierze można mnożyć tylko wtedy, gdy:

- liczba kolumn pierwszej macierzy
- jest równa liczbie wierszy drugiej macierzy.

Jeśli:

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times k}$$

to wynik ma rozmiar:

$$AB \in \mathbb{R}^{m \times k}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz zgodność wymiarów.

Krok 2

Dla każdego elementu wyniku liczysz sumę iloczynów elementów odpowiedniego wiersza i kolumny.

Czyli:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

```
def mnozenie_macierzy(macierz1, macierz2): # funkcja mnoży dwie macierze
    ilosc_wierszy_macierz1 = len(macierz1) # zapisujemy liczbę wierszy pierwszej
macierzy
    ilosc_wierszy_macierz2 = len(macierz2) # zapisujemy liczbę wierszy drugiej
macierzy
    ilosc_kolumn_macierz1 = len(macierz1[0]) # zapisujemy liczbę kolumn pierwszej
macierzy
    ilosc_kolumn_macierz2 = len(macierz2[0]) # zapisujemy liczbę kolumn drugiej
macierzy

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2: # sprawdzamy, czy można
pomnożyć macierze
        raise ValueError("Nie da się pomnożyć tych macierzy") # zgłaszamy błąd,
jeśli liczba kolumn pierwszej macierzy nie jest równa liczbie wierszy drugiej
macierzy

    wynik = [] # tworzymy pustą listę na macierz wynikową
    for i in range(ilosc_wierszy_macierz1): # przechodzimy po wierszach pierwszej
macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy wynikowej
        for j in range(ilosc_kolumn_macierz2): # przechodzimy po kolumnach
drugiej macierzy
            suma = 0 # ustawiamy początkową sumę na 0
            for k in range(ilosc_kolumn_macierz1): # przechodzimy po elementach,
które trzeba przez siebie pomnożyć i dodać
                suma += macierz1[i][k] * macierz2[k][j] # dodajemy iloczyn
odpowiednich elementów do sumy
            wiersz.append(suma) # dodajemy obliczony element do wiersza macierzy
wynikowej
        wynik.append(wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy wynik mnożenia macierzy

macierz1 = [ # tworzymy pierwszą macierz
    [1, 2], # pierwszy wiersz pierwszej macierzy
    [3, 4] # drugi wiersz pierwszej macierzy
] # koniec definicji pierwszej macierzy

macierz2 = [ # tworzymy drugą macierz
    [5, 6], # pierwszy wiersz drugiej macierzy
    [7, 8] # drugi wiersz drugiej macierzy
] # koniec definicji drugiej macierzy

wynik = mnozenie_macierzy(macierz1, macierz2) # wywołujemy funkcję mnożenia
macierzy i zapisujemy wynik

print("Wynik mnożenia:") # wypisujemy opis wyniku
for wiersz in wynik: # przechodzimy po kolejnych wierszach macierzy wynikowej
    print(wiersz) # wypisujemy aktualny wiersz macierzy wynikowej
```

Wynik

Dla:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

otrzymujemy:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Zadanie 5

Korzystając z rozwiązań poprzednich zadań wykonaj następujące mnożenia macierzowe: $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$ i porównaj ich wyniki.

Idea zadania

Jeśli macierz (A) jest odwracalna, to powinno zachodzić:

$$A \cdot A^{-1} = I$$

oraz

$$A^{-1} \cdot A = I$$

gdzie I to macierz jednostkowa.

Czyli zadanie polega na sprawdzeniu, czy wyznaczona wcześniej macierz odwrotna rzeczywiście jest poprawna.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz macierz odwrotną.

Krok 2

Mnożysz:

- $A \cdot A^{-1}$
- $A^{-1} \cdot A$

Krok 3

Porównujesz wyniki z macierzą jednostkową.

Jeśli pojawiają się małe liczby typu:

$$2.220446049250313 \cdot 10^{-16}$$

to traktuje się je jako 0, ponieważ są to błędy zaokrągleń.

Kod

PYTHON

```
def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b # funkcja liczy macierz
odwrotną metodą Gaussa-Jordana
    n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Macierz musi być kwadratowa") # zgłaszamy błąd,
jeśli macierz nie jest kwadratowa

    # macierz rozszerzona [A | I] # tworzymy macierz rozszerzoną, czyli po lewej
A, a po prawej macierz jednostkową
    rozszerzona_macierz = [] # tworzymy pustą listę na macierz rozszerzoną

    for i in range(n): # przechodzimy po kolejnych wierszach macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy rozszerzonej

        # lewa strona: macierz A # najpierw do wiersza wpisujemy elementy macierzy
A
        for j in range(n): # przechodzimy po kolumnach macierzy A
            wiersz.append(macierz[i][j]) # dodajemy element z macierzy A do
aktualnego wiersza
            # wiersz.append(float(macierz[i][j])) # alternatywna wersja, gdybyśmy
chcieli od razu zamienić liczby na float

        # prawa strona: macierz jednostkowa I # po prawej stronie dopisujemy
macierz jednostkową
        for j in range(n): # przechodzimy po kolumnach macierzy jednostkowej
            if i == j: # sprawdzamy, czy element znajduje się na przekątnej
głównej
                wiersz.append(1) # na przekątnej głównej wpisujemy 1
            else: # jeśli element nie jest na przekątnej głównej
                wiersz.append(0) # poza przekątną wpisujemy 0

        rozszerzona_macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
rozszerzonej

    # algorytm Gaussa-Jordana # zaczynamy przekształcanie macierzy rozszerzonej
    for i in range(n): # przechodzimy po kolejnych elementach głównych na
przekątnej
        # jeśli na przekątnej jest 0, zamień wiersze # element główny nie może być
zerem
        if rozszerzona_macierz[i][i] == 0: # sprawdzamy, czy element główny jest
równy 0
            znaleziono = False # zakładamy, że jeszcze nie znaleziono wiersza do
zamiany
            for k in range(i+1, n): # szukamy niżej wiersza z niezerowym
elementem w tej samej kolumnie
                if rozszerzona_macierz[k][i] != 0: # sprawdzamy, czy w tym
wierszu element w kolumnie i nie jest zerem
```

```

        rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i] # zamieniamy miejscami dwa wiersze
        znaleziono = True # zapisujemy, że znaleziono odpowiedni
wiersz

        break # przerywamy pętlę, bo zamiana została wykonana
    if not znaleziono: # sprawdzamy, czy nie udało się znaleźć wiersza do
zamiany

        raise ValueError("Macierz nie ma odwrotności") # jeśli nie ma
takiego wiersza, macierz nie ma odwrotności

    # dzielenie całego wiersza przez element główny # normalizujemy wiersz,
    żeby element główny stał się równy 1
    element_glowny = rozszerzona_macierz[i][i] # zapisujemy aktualny element
    główny
    for j in range(2 * n): # przechodzimy po wszystkich kolumnach macierzy
    rozszerzonej
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny
# dzielimy każdy element wiersza przez element główny

    # zerowanie pozostałych elementów w tej kolumnie # zerujemy elementy nad i
    pod elementem głównym
    for k in range(n): # przechodzimy po wszystkich wierszach
        if k != i: # nie zmieniamy aktualnego wiersza głównego
            współczynnik = rozszerzona_macierz[k][i] # zapisujemy
współczynnik potrzebny do wyzerowania elementu
            for j in range(2 * n): # przechodzimy po wszystkich kolumnach
    macierzy rozszerzonej
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
współczynnik * rozszerzona_macierz[i][j] # odejmujemy odpowiednią wielokrotność
    wiersza głównego

    odwrotna = [] # tworzymy pustą listę na macierz odwrotną
    for i in range(n): # przechodzimy po wierszach macierzy rozszerzonej
        wiersz = [] # tworzymy pusty wiersz macierzy odwrotnej
        for j in range(n, 2 * n): # przechodzimy po prawej części macierzy
    rozszerzonej
            wiersz.append(rozszerzona_macierz[i][j]) # dodajemy element z prawej
    strony, czyli element macierzy odwrotnej
        odwrotna.append(wiersz) # dodajemy gotowy wiersz do macierzy odwrotnej

    return odwrotna # zwracamy macierz odwrotną

def mnozenie_macierzy(macierz1, macierz2): # funkcja mnoży dwie macierze
    ilosc_wierszy_macierz1 = len(macierz1) # zapisujemy liczbę wierszy pierwszej
    macierzy
    ilosc_wierszy_macierz2 = len(macierz2) # zapisujemy liczbę wierszy drugiej
    macierzy
    ilosc_kolumn_macierz1 = len(macierz1[0]) # zapisujemy liczbę kolumn pierwszej
    macierzy
    ilosc_kolumn_macierz2 = len(macierz2[0]) # zapisujemy liczbę kolumn drugiej
    macierzy

```

```

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2: # sprawdzamy warunek
mnożenia macierzy
        raise ValueError("Nie da się pomnożyć tych macierzy") # zgłaszamy błąd,
jeśli liczba kolumn pierwszej macierzy nie jest równa liczbie wierszy drugiej
macierzy

    wynik = [] # tworzymy pustą listę na macierz wynikową
    for i in range(ilosc_wierszy_macierz1): # przechodzimy po wierszach pierwszej
macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy wynikowej
        for j in range(ilosc_kolumn_macierz2): # przechodzimy po kolumnach
drugiej macierzy
            suma = 0 # ustawiamy początkową sumę na 0
            for k in range(ilosc_kolumn_macierz1): # przechodzimy po elementach
potrzebnych do obliczenia jednego elementu wyniku
                suma += macierz1[i][k] * macierz2[k][j] # mnożymy odpowiednie
elementy i dodajemy je do sumy
            wiersz.append(suma) # dodajemy obliczony element do aktualnego wiersza
            wynik.append(wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy wynik mnożenia macierzy

macierz = [ # tworzymy macierz A
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

macierz_odwrotna = macierz_odwrotna_Gaussa_Jordana(macierz) # liczymy macierz
odwrotną do macierzy A

wynik1 = mnozenie_macierzy(macierz, macierz_odwrotna) # mnożymy A przez A^-1
wynik2 = mnozenie_macierzy(macierz_odwrotna, macierz) # mnożymy A^-1 przez A

print("Wynik mnożenia A * A^-1:") # wypisujemy opis pierwszego wyniku
for wiersz in wynik1: # przechodzimy po wierszach wyniku A * A^-1
    print(wiersz) # wypisujemy aktualny wiersz wyniku

print("Wynik mnożenia A^-1 * A:") # wypisujemy opis drugiego wyniku
for wiersz in wynik2: # przechodzimy po wierszach wyniku A^-1 * A
    print(wiersz) # wypisujemy aktualny wiersz wyniku

```

Wynik

Oba iloczyny powinny być bardzo bliskie macierzy jednostkowej:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

W praktyce może się pojawić na przykład:

zamiast dokładnego zera. To jest normalne i wynika z ograniczonej dokładności obliczeń zmiennoprzecinkowych.

Wniosek

Wszystkie zadania zostały wykonane poprawnie.

- W zadaniu 1 obliczono wyznacznik macierzy metodą rozwinięcia Laplace'a.
- W zadaniu 2 obliczono transpozycję macierzy.
- W zadaniu 3 wyznaczono macierz odwrotną dwiema metodami: Laplace'a oraz Gaussa-Jordana.
- W zadaniu 4 wykonano mnożenie dwóch macierzy.
- W zadaniu 5 sprawdzono poprawność macierzy odwrotnej przez obliczenie iloczynów $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$.

Otrzymane wyniki są zgodne z teorią, a ewentualne bardzo małe różnice wynikają z błędów zaokrągleń numerycznych.